

SQL DATA ANALYSIS CASE STUDY

Flipkart Sales & Profitability Analysis Using SQL

BY ARAVIND RADHAKRISHNAN

1. Objective of the Case Study

The objective of this case study is to analyze Flipkart sales data using SQL and extract meaningful business insights.

- Analyze overall sales and profitability.
- Identify high-performing products and categories.
- Analyze regional performance and payment methods.
- Study sales trends over time.
- Compare product sales with average product sales.
- Rank products using window functions.
- Segment products into sales groups using NTILE().
- Demonstrate SQL skills from basic queries to advanced CTEs, subqueries, and window functions.

2. About the Dataset

The dataset contains e-commerce sales transaction records from Flipkart. Each record contains information about the order, product, category, price, quantity, sales amount, order date, payment method, customer rating, profit, discount, customer segment, and region.

The dataset can be used to analyze sales performance, product performance, category performance, profitability, customer segments, regional sales, payment methods, discounts, customer ratings, and monthly sales trends.

2.1 Column Description

Column Name	Description
Order ID	Identifier associated with an order
Product Name	Name of the product sold
Category	Product category
Price (INR)	Price of the product in Indian Rupees
Quantity Sold	Number of units sold
Total Sales (INR)	Total sales generated from the transaction
Order Date	Date on which the order was placed
Payment Method	Payment method used for the transaction
Customer Rating	Rating given by the customer

Month	Month associated with the order
Year	Year associated with the order
Profit (INR)	Profit generated from the transaction
Discount %	Discount percentage applied
Customer Segment	Customer segment associated with the order
Region	Geographic region of the order

3. Database Setup

The analysis was performed using MySQL Workbench.

USE ecommerce;

Analysis table:

flipkart_sales_enriched

1. Retrieve all records

```
SELECT *
FROM flipkart_sales_enriched;
```

2. Count the total number of records

```
SELECT COUNT(*) AS Total_Rows
FROM flipkart_sales_enriched;
```

3. Display the table structure

```
DESCRIBE flipkart_sales_enriched;
```

4. Find the date range

```
SELECT MIN(`Order Date`) AS First_Order_Date,
       MAX(`Order Date`) AS Last_Order_Date
FROM flipkart_sales_enriched;
```

5. Identify repeated transaction records

```
SELECT `Order ID`, `Product Name`, `Order Date`,
       `Total Sales (INR)`, COUNT(*) AS Duplicate_Count
FROM flipkart_sales_enriched
GROUP BY `Order ID`, `Product Name`, `Order Date`, `Total Sales (INR)`
```

```
HAVING COUNT(*) > 1  
ORDER BY Duplicate_Count DESC;
```

6. Find orders appearing multiple times

```
SELECT `Order ID`, COUNT(*) AS Number_of_Rows  
FROM flipkart_sales_enriched  
GROUP BY `Order ID`  
HAVING COUNT(*) > 1  
ORDER BY Number_of_Rows DESC;
```

7. Find all unique categories

```
SELECT DISTINCT Category  
FROM flipkart_sales_enriched;
```

8. Find all unique regions

```
SELECT DISTINCT Region  
FROM flipkart_sales_enriched;
```

9. Find all unique payment methods

```
SELECT DISTINCT `Payment Method`  
FROM flipkart_sales_enriched;
```

10. List all unique products

```
SELECT DISTINCT `Product Name`  
FROM flipkart_sales_enriched  
ORDER BY `Product Name`;
```

11. Calculate total sales

```
SELECT ROUND(SUM(`Total Sales (INR)`), 2) AS Total_Sales  
FROM flipkart_sales_enriched;
```

Purpose: Calculate the total sales generated across all transactions.

12. Calculate total profit

```
SELECT ROUND(SUM(`Profit (INR)`), 2) AS Total_Profit  
FROM flipkart_sales_enriched;
```

Purpose: Determine the total profit generated from all transactions.

13. Calculate average sales per transaction

```
SELECT ROUND(AVG(`Total Sales (INR)`), 2) AS Average_Sales  
FROM flipkart_sales_enriched;
```

Purpose: Determine the average sales value of a transaction.

14. Calculate total quantity sold

```
SELECT SUM(`Quantity Sold`) AS Total_Quantity_Sold  
FROM flipkart_sales_enriched;
```

15. Find products with the highest total sales

```
SELECT `Product Name`,  
       ROUND(SUM(`Total Sales (INR)`), 2) AS Total_Sales  
FROM flipkart_sales_enriched  
GROUP BY `Product Name`  
ORDER BY Total_Sales DESC;
```

Purpose: Identify products that generate the highest total sales.

16. Find total sales by category

```
SELECT Category,  
       ROUND(SUM(`Total Sales (INR)`), 2) AS Total_Sales  
FROM flipkart_sales_enriched  
GROUP BY Category  
ORDER BY Total_Sales DESC;
```

17. Find total profit by category

```
SELECT Category,  
       ROUND(SUM(`Profit (INR)`), 2) AS Total_Profit  
FROM flipkart_sales_enriched  
GROUP BY Category  
ORDER BY Total_Profit DESC;
```

18. Find products whose sales are above the average product sales

```
SELECT `Product Name`,  
       SUM(`Total Sales (INR)`) AS Total_Sales  
FROM flipkart_sales_enriched  
GROUP BY `Product Name`  
HAVING SUM(`Total Sales (INR)`) >  
(
```

```
SELECT AVG(Product_Sales)
FROM
(
    SELECT `Product Name`,
           SUM(`Total Sales (INR)`) AS Product_Sales
    FROM flipkart_sales_enriched
    GROUP BY `Product Name`
) AS ProductSummary
);
```

19. Rank products according to total sales

```
WITH ProductSales AS
(
    SELECT `Product Name`,
           SUM(`Total Sales (INR)`) AS Total_Sales
    FROM flipkart_sales_enriched
    GROUP BY `Product Name`
)
SELECT `Product Name`,
       ROUND(Total_Sales, 2) AS Total_Sales,
       RANK() OVER
       (ORDER BY Total_Sales DESC) AS Sales_Rank
FROM ProductSales;
```

20. Compare monthly sales with the previous month

```
WITH MonthlySales AS
(
    SELECT YEAR(`Order Date`) AS Year,
           MONTH(`Order Date`) AS Month,
           SUM(`Total Sales (INR)`) AS Total_Sales
    FROM flipkart_sales_enriched
    GROUP BY YEAR(`Order Date`), MONTH(`Order Date`)
)
SELECT Year,
       Month,
       ROUND(Total_Sales, 2) AS Total_Sales,
       ROUND(
           LAG(Total_Sales) OVER
           (ORDER BY Year, Month), 2
```

```
    ) AS Previous_Month_Sales  
FROM MonthlySales  
ORDER BY Year, Month;
```

21. Divide products into three sales groups

```
WITH ProductSales AS  
(  
    SELECT `Product Name`,  
           SUM(`Total Sales (INR)`) AS Total_Sales  
    FROM flipkart_sales_enriched  
    GROUP BY `Product Name`  
)  
SELECT `Product Name`,  
       ROUND(Total_Sales, 2) AS Total_Sales,  
       NTILE(3) OVER  
       (ORDER BY Total_Sales) AS Sales_Group  
FROM ProductSales;
```

Tools Used

- MySQL
- MySQL Workbench
- Kaggle Dataset

End of Case Study